# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

## Department of Computer Science and Engineering

## CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

| | | | |
|---|---|---|---|
| **Course Code:** | CSA10 / CSA1063 | **Course Title:** | Software Engineering |
| **CO Assessed:** | CO2 | **Assessment:** | Assessment Tool 2 – Scenario-Based Requirement Analysis |
| **Weightage:** | 20% | **Total Marks:** | 25 |
| **CO2 Statement:** | *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills* | | |
| **Student Name:** | HARI VISHNU N | **Reg. No.:** | 192524319 |
| **Date:** | 21-10-26 | **Duration:** | 60 minutes |

### Code of Conduct

*I HARI VISHNU N / 192524319 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

## Annexure A – Scenario-Based Requirement Analysis

**Instructions to Students:**

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

**Question**

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description
   - Scope of the System
   - Functional Requirements

- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

## ASSIGNED PROBLEM STATEMENT:

### 11. AI-Powered Requirements Conflict and Ambiguity Detection System

**Industry Context:**

Large software projects receive requirements from users, customers, business teams, developers, security teams, and regulators. Requirements may be incomplete, ambiguous, duplicated, or contradictory. These problems can remain hidden until design or implementation, resulting in costly rework.

**Problem Statement:**

Design and develop an AI-Powered Requirements Conflict and Ambiguity Detection System that analyses software requirements and identifies inconsistencies, ambiguity, duplication, missing dependencies, and conflicting stakeholder expectations.

The system should support collaborative documentation, requirements prioritization, traceability, and human validation. It should integrate with project-management tools and maintain version history while ensuring that AI recommendations remain reviewable and explainable.

**Objectives:**

The system should:

- Analyse requirements for ambiguity and contradictions
- Detect duplicate and conflicting requirements
- Identify missing dependencies and incomplete information
- Support MoSCoW or similar prioritization techniques
- Maintain traceability between requirements and development tasks
- Provide collaborative review and version control
- Support explainable and human-validated AI recommendations

**Expected Outcomes**:

The proposed system will:

- Reduce requirements-related rework
- Improve clarity and consistency of software specifications
- Support better stakeholder collaboration
- Improve requirements traceability
- Reduce project risks caused by conflicting requirements
- Strengthen requirements engineering practices

## 1. Study and Analyze the Scenario

**Question:**

- Carefully understand the given problem statement.
- Identify the business domain, stakeholders, existing challenges, and system objectives.

**Answer:**

### 1.1 Business Domain

The scenario belongs to requirements engineering and software project management. The proposed system, named ReqGuard AI for this analysis, sits between requirement capture and system design. It acts as an automated quality gate that examines natural-language requirements before they are converted into designs and code, which matters most in large projects where requirements arrive from many sources.

### 1.2 Stakeholders

- **Business analysts:** elicit, document, and maintain requirements.
- **Product owners and project managers:** prioritize requirements and track delivery.
- **Developers and architects:** design and implement features from the requirements.
- **QA engineers:** derive test cases and verify acceptance criteria.
- **Security team:** defines security and privacy requirements.
- **Regulators and compliance officers:** define legal obligations that must be traceable.
- **End users and customers:** originate business needs and validate outcomes.
- **System administrators:** manage accounts, roles, and integrations.

### 1.3 Existing Challenges

- Vague wording such as “fast”, “user-friendly”, or “as appropriate” makes requirements impossible to test.
- Duplicate requirements raised by different teams cause wasted effort and version confusion.
- Contradictions, such as offline access requested alongside a cloud-only data rule, surface only during design or testing.
- Missing dependencies and acceptance criteria force developers to make assumptions.
- Manual reviews are slow, inconsistent, and do not scale with requirement volume.
- Weak traceability between requirements, tasks, and tests makes impact analysis difficult.

### 1.4 System Objectives

The system must analyse requirements for ambiguity, contradiction, duplication, and missing dependencies; support MoSCoW prioritization; maintain traceability to development tasks; provide collaborative review and version control; and present every AI recommendation in an explainable form that a human can accept or reject. The expected outcome is a measurable reduction in rework caused by specification defects.

## 2.Identify Functional and Non-Functional Requirements

**Question:**

- List all functional requirements required by the proposed system.
- Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.

**Answer:**

### 2.1 Functional Requirements

| ID | Requirement | Description |
|---|---|---|
| FR-01 | Requirement import and management | Import from DOCX, CSV, and Excel; create, edit, and store requirements with unique IDs. |
| FR-02 | Ambiguity detection | Flag vague terms (e.g., “fast”, “easy”), unresolved pronouns, and requirements without measurable criteria. |
| FR-03 | Contradiction detection | Identify pairs of requirements with opposing constraints using semantic analysis. |
| FR-04 | Duplicate detection | Cluster near-duplicate requirements using semantic similarity above a configurable threshold. |
| FR-05 | Dependency and completeness checks | Detect references to undefined actors, features, or data, and missing acceptance criteria. |
| FR-06 | MoSCoW prioritization | Allow tagging as Must, Should, Could, or Won’t; AI suggests a tag with justification. |
| FR-07 | Traceability | Link requirements to user stories, tasks, design elements, and test cases; generate a traceability matrix. |
| FR-08 | Stakeholder conflict view | Show differing expectations from stakeholder groups about the same feature. |
| FR-09 | Collaborative review | Comments, @mentions, assignment, and status workflow (Draft, Under Review, Approved). |
| FR-10 | Version control | Store each change with author and timestamp; compare versions and restore earlier ones. |
| FR-11 | Explainability | Show, for each AI finding, the evidence sentences, the rule or model reasoning, and a confidence score. |
| FR-12 | Human validation | Reviewers accept, reject, or modify each recommendation; every decision is logged. |

| | | |
|---|---|---|
| FR-13 | PM tool integration | Synchronize requirements and tasks with Jira and Azure DevOps through their APIs. |
| FR-14 | Reporting | Dashboard showing quality score, open conflicts, and priority distribution; export to PDF and CSV. |
| FR-15 | User and role management | Create users, assign roles (RBAC), and manage project permissions. |
| FR-16 | Audit log export | Export the complete change and decision history for compliance review. |

## 2.2 Non-Functional Requirements

| Category | Requirement | Measurable Target |
|---|---|---|
| Performance | Analysis of a requirement set | Analyse 1,000 requirements in 2 minutes or less |
| Performance | Interactive response | Pages load within 2 seconds |
| Security | Access control and authentication | Role-based access; SSO and MFA supported |
| Security | Data protection | TLS 1.2 or higher in transit; AES-256 encryption at rest |
| Reliability | Data integrity | No loss of approved versions or review decisions; daily backups |
| Availability | Uptime | 99.5% or higher monthly |
| Usability | Accessibility and learnability | WCAG 2.1 AA compliance; new user productive within 1 hour |
| Scalability | Load capacity | 50 concurrent users; 100,000 requirements per project |
| Maintainability | Code and interface quality | Modular services; 80% or higher unit test coverage; documented APIs |
| Accuracy | Ambiguity and conflict detection | Precision of 85% or higher on the validation dataset |
| Explainability | Transparency of AI output | 100% of AI findings show evidence and confidence |
| Compliance | Audit retention | Audit logs retained for a configurable period (default 3 years) |

## Question 3: Identify Actors and Use Cases

**Question:**

- Identify all primary and secondary actors interacting with the system.
- List the major use cases associated with each actor.
- Draw a Use Case Diagram representing the interactions between actors and the system.

**Answer:**

### 3.1 Actors

- **Primary actors:** Business Analyst, Product Owner / Project Manager, Developer, Stakeholder (customer or business user), and QA Reviewer.
- **Secondary actors:** Security Officer, Compliance Officer, System Administrator, and the External PM Tool (Jira / Azure DevOps), which acts as a system actor.

### 3.2 Major Use Cases by Actor

| Use Case | Description | Primary Actor(s) |
|---|---|---|
| UC-01 | Import requirements | Business Analyst |
| UC-02 | Detect ambiguity and missing information | Business Analyst |
| UC-03 | Detect conflicts and duplicates | Business Analyst, Security Officer |
| UC-04 | Prioritize requirements (MoSCoW) | Product Owner / PM |
| UC-05 | Link requirements to tasks (traceability) | Developer, Product Owner / PM |
| UC-06 | Collaborate and comment | Stakeholder, Developer |
| UC-07 | Validate AI findings | QA Reviewer, Business Analyst |
| UC-08 | Manage version history | QA Reviewer, Compliance Officer |
| UC-09 | View explanations of AI findings | Developer |
| UC-10 | Generate reports | Product Owner / PM |
| UC-11 | Synchronize with PM tools | External PM Tool |
| UC-12 | Manage users and roles | System Administrator |

## 3.3 Use Case Diagram

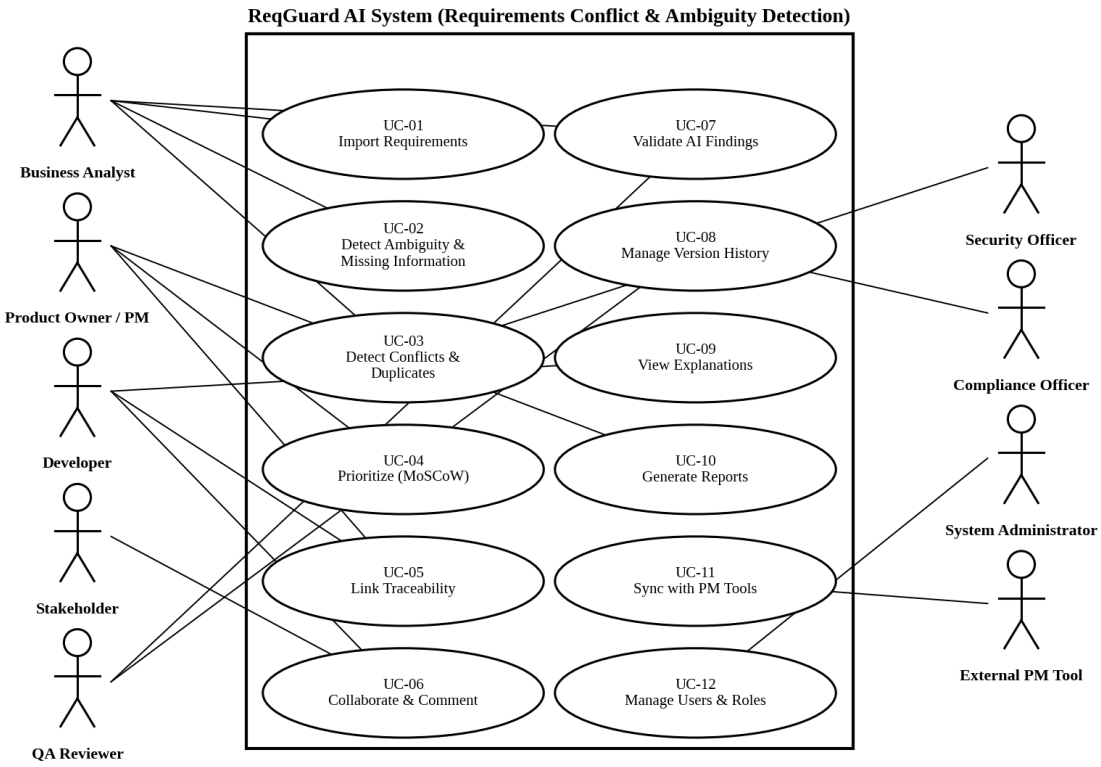



**Figure 1: Use Case Diagram of the ReqGuard AI System**

## 4. Prepare the Software Requirement Specification (SRS)

**Question:**

Develop an SRS document containing: Introduction; Problem Description; Scope of the System; Functional Requirements; Non-Functional Requirements; Use Case Descriptions; Data Requirements; External Interface Requirements; and System Features.

**Answer:**

### 4.1 Introduction

This SRS specifies the requirements of ReqGuard AI, an AI-powered requirements conflict and ambiguity detection system. It is intended for developers, testers, project managers, and stakeholders, and follows the structure recommended by IEEE/ISO/IEC 29148. Key terms: an AI finding is a machine-generated observation about a requirement; human validation is a reviewer's accept, reject, or modify decision; MoSCoW denotes Must, Should, Could, and Won't priorities.

### 4.2 Problem Description

Requirements in large projects are frequently incomplete, ambiguous, duplicated, or contradictory. Defects found during design or implementation cost far more to fix than those caught early. The system addresses this by automating the first-pass analysis and keeping humans in control of every decision (see Question 1).

### 4.3 Scope of the System

**In scope:** requirement import, quality analysis, conflict and duplicate detection, dependency checks, MoSCoW prioritization, traceability, collaborative review, version history, explainable AI findings, PM tool synchronization, and reporting.

**Out of scope:** requirement elicitation interviews, automatic code generation, and automated test execution.

### 4.4 Functional and Non-Functional Requirements

The functional requirements FR-01 to FR-16 and the non-functional requirements (performance, security, reliability, availability, usability, scalability, maintainability, accuracy, explainability, and compliance) are specified in full in Question 2, Sections 2.1 and 2.2.

### 4.5 Use Case Descriptions

The table below gives the detailed description of UC-02; the other use cases in Question 3 follow the same template.

| Field | Detail |
|---|---|
| Use Case ID / Name | UC-02 / Detect Ambiguity and Missing Information |
| Primary actor | Business Analyst |
| Preconditions | Requirements have been imported into a project. |
| Main flow | 1. User selects a requirement set. 2. System analyses each requirement. 3. System flags vague terms, unmeasurable criteria, and undefined references. 4. System shows each finding with its evidence and a suggested rewrite. |
| Alternate flow | If no issue is found, the requirement is marked “Clear” and the check is recorded. |
| Postconditions | Findings are stored and wait for human validation (UC-07). |

### 4.6 Data Requirements

- **Requirement:** ID, title, description, type, MoSCoW priority, status, version, owner, and project.
- **Finding:** type (ambiguity, conflict, duplicate, missing dependency), linked requirement IDs, confidence score, explanation, and reviewer decision.
- **Stakeholder and user:** name, role, contact, and access rights.
- **Traceability link:** requirement ID, task or test ID, and link type.
- **Version and audit record:** snapshot, author, timestamp, and action performed.

### 4.7 External Interface Requirements

- **User interface:** responsive web application usable in modern browsers.
- **Software interfaces:** REST/JSON APIs of Jira and Azure DevOps; DOCX, CSV, and Excel import.
- **AI service interface:** secure API to a language model or embedding service for analysis.
- **Authentication:** SSO through SAML or OAuth 2.0; LDAP directory support.
- **Communication:** HTTPS for all client-server and integration traffic; email notifications.

### 4.8 System Features

- F1: Requirement quality analysis (ambiguity, completeness).
- F2: Conflict and duplicate detection.
- F3: MoSCoW prioritization with human override.
- F4: Traceability to tasks and tests.
- F5: Collaborative review and workflow.

- F6: Version control and audit history.
- F7: Explainable AI findings and human validation.
- F8: Integration with PM tools and reporting.

## 5. Identify Assumptions and Constraints

**Question:**

- List the assumptions considered while developing the system.
- Identify technical, operational, economic, legal, and time-related constraints affecting the project.

**Answer:**

### 5.1 Assumptions

- Requirements are written in English as natural-language sentences.
- Stakeholders have web browser access and will take part in review cycles.
- Jira and Azure DevOps provide documented REST APIs with access tokens.
- A language model or embedding service is available through a cloud API or local hosting.
- Project teams accept that AI findings are advisory and that people make the final decisions.
- Requirement documents contain no data that organizational policy forbids sending to a cloud AI service.

### 5.2 Constraints

| Type | Constraint | Mitigation |
|---|---|---|
| Technical | NLP accuracy is limited for domain jargon and very short requirements. | Use a project glossary; keep human validation mandatory. |
| Operational | Teams must learn a new review workflow; adoption depends on training. | Role-based onboarding and guided tutorials. |
| Economic | AI API and cloud hosting costs grow with requirement volume. | Analyse only changed requirements; cache results; set budget limits. |
| Legal | Client requirements may be confidential; GDPR and data-residency rules apply. | Encryption, tenant isolation, regional hosting, and data processing agreements. |
| Time-related | The academic project schedule limits scope to a minimum viable product. | Deliver core analysis first, then integrations and dashboards. |

## 6. Validate Requirement Completeness and Consistency

**Question:**

- Verify whether all stakeholder requirements have been addressed.
- Check for ambiguity, redundancy, inconsistency, and missing requirements.
- Suggest improvements to enhance the quality and completeness of the requirement specification.

**Answer:**

### 6.1 Coverage of Stakeholder Requirements

| Stakeholder Group | Addressed By | Status |
|---|---|---|
| Business analysts | FR-01, FR-02, FR-03, FR-05; UC-01, UC-02 | Covered |
| Product owners / PMs | FR-06, FR-07, FR-14; UC-04, UC-10 | Covered |
| Developers | FR-07, FR-11; UC-05, UC-09 | Covered |
| QA engineers | FR-07 (test links), FR-12; UC-07, UC-08 | Covered; test-case links to be explicit |
| Security team | FR-03 (security conflicts); security NFRs | Covered |
| Compliance officers | FR-10, FR-16; audit retention NFR | Covered after adding FR-16 |
| End users / customers | FR-08, FR-09; UC-06 | Covered |
| System administrators | FR-13, FR-15; UC-11, UC-12 | Covered |

### 6.2 Quality Check of the Specification

| Issue Type | Finding in the Draft | Resolution |
|---|---|---|
| Ambiguity | "Fast response" and "explainable" were undefined in early drafts. | Quantified (pages load within 2 seconds); explainability defined in FR-11. |
| Redundancy | FR-02 and FR-05 both check for incomplete information. | FR-02 covers wording and measurability; FR-05 covers dependencies only. |
| Inconsistency | FR-06 could auto-assign priorities, conflicting with the human-validation principle. | AI only suggests; the human makes the final MoSCoW decision. |
| Inconsistency | A 95% precision target conflicted with limited training data at launch. | Phased target: 85% at the minimum viable product, improved through feedback. |
| Missing | No data retention, backup, or disaster recovery requirement. | Added to reliability NFRs and data requirements. |

| | | |
|---|---|---|
| Missing | No audit log export and no multilingual input requirement. | FR-16 added for audit export; multilingual support listed as future scope. |

### 6.3 Suggested Improvements

- Add measurable acceptance criteria to every functional requirement before baseline approval.
- Create a shared glossary of domain terms to reduce ambiguity across teams.
- Link a formal change-control process to the version history feature.
- Pilot the MoSCoW and review workflow with real stakeholders to confirm usability.
- Repeat this validation after every baseline change to keep the specification consistent.

### Rubrics for Calculation

| Criteria | Excellent (4 Marks) | Good (3 Marks) | Satisfactory (2 Marks) | Needs Improvement (1 Mark) |
|---|---|---|---|---|
| **Scenario Understanding and Problem Analysis (30%)** | Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis. | Demonstrates good understanding with minor omissions. | Basic understanding with limited analysis. | Poor understanding; major aspects missing or incorrect. |
| **Functional and Non-Functional Requirement Identification (25%)** | Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements. | Identifies most requirements with minor classification errors. | Identifies only basic requirements; several omissions. | Requirements are incomplete, inaccurate, or poorly classified. |
| **Actor Identification and Use Case Modelling (20%)** | Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards. | Most actors and use cases identified; diagram has minor errors. | Limited actors/use cases identified; diagram partially correct. | Actors/use cases missing or incorrect; diagram absent or inaccurate. |
| **Software Requirement Specification (SRS) Documentation (15%)** | SRS is well-structured, complete, professionally organized, and includes all required sections. | SRS is organized with minor missing sections or formatting issues. | SRS includes only essential sections with limited detail. | SRS is incomplete, poorly organized, or lacks essential components. |
| **Assumptions, Constraints, and Requirement Validation (10%)** | Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification. | Identifies most assumptions and constraints; validation is adequate. | Limited assumptions/constraints; validation is superficial. | Assumptions, constraints, and validation are missing or incorrect. |

| Criteria | **Mark** |
|---|---|
| Scenario Understanding and Problem Analysis (30%) | /5 |
| Functional and Non-Functional Requirement Identification (25%) | /5 |
| Actor Identification and Use Case Modelling (20%) | /5 |
| Software Requirement Specification (SRS) Documentation (15%) | /5 |
| Assumptions, Constraints, and Requirement Validation (10%) | /5 |
| **TOTAL** | /25 |